



Report Project **MINI Project**

I3-AMS-A-Group 3 (2024-2025)

TOPIC: Prediction Traffic

Members

Name of student	ID of student	Scores
1. DOEUN Bunheng	e20221528	
2. CHHO Kosol	e20220564	
3. CHIV Mengchou	e20221028	
4. CHHENG Sothean	e20220686	

Lecturer

Mr. Malai NHIM

Contents

1	Abstract	3
1.1	Introduction	3
1.2	Objective	4
2	Methodology	5
2.1	Data-description	6
2.2	Exploratory-data-analysis	7
2.2.1	Data Cleaning	8
2.2.2	Visualizing the data transformation	10
	Encoding Categorical Variables:	11
3	Model Building	13
3.1	Model Arima and Sarima	14
3.2	XGBoost and advantage XGBoost	17
3.3	Random-forest Model	19
3.4	LSTM Model deep learning	21
	Here is graph that predicted by using model LSTM	22
3.5	GRU Model	22
	work-flow of GRU Model	23
4	Evaluation and Result	25
4.1	Reference	26

Chapter 1

Abstract

Traffic congestion is a major challenge in urban areas, leading to increased travel times, fuel consumption, and environmental pollution. This study aims to enhance traffic management by analyzing a comprehensive dataset of hourly vehicle counts collected from various junctions. Using data analysis and machine learning techniques, we explore traffic patterns such as peak traffic hours, daily and seasonal fluctuations, and differences across different junctions. Our results show that these insights can significantly improve traffic prediction accuracy and support better decision-making for urban planners, helping reduce congestion and improve city transportation systems.

1.1 Introduction

Traffic congestion is one of the most pressing problems faced by urban areas today. It causes longer travel times, higher fuel costs, and contributes to air pollution, negatively impacting both the economy and quality of life. Despite efforts to improve traffic management, the complexity of urban traffic systems requires more effective and data-driven solutions. The objective of this study is to better understand traffic flow patterns using a large dataset that includes hourly vehicle counts from several junctions. By analyzing this data, we aim to address key research questions: What are the peak traffic hours and how do they vary across different junctions? How do daily and seasonal patterns impact traffic congestion? Can data analysis and machine learning improve the accuracy of traffic predictions? This research is important as it offers potential solutions to the widespread

issue of traffic congestion, which affects cities worldwide. By providing a clearer understanding of traffic behaviors, this study can contribute to smarter urban planning, better resource allocation, and more sustainable transportation systems.

1.2 Objective

In this project, I will be exploring the dataset of four junctions and built a model to predict traffic on the same. This could potentially help in solving the traffic congestion problem by providing a better understanding of traffic patterns that will further help in building an infrastructure to eliminate the problem. The primary objectives of this project are:

- +To analyze hourly vehicle traffic patterns across multiple junctions using time-series data.

- +To identify peak traffic hours and high-congestion junctions to better understand urban traffic flow behavior.

- +To visualize traffic trends over different days, weeks, and months for clearer interpretation.

- +To explore potential applications of traffic data for urban planning in Cambodian cities.

- +To propose data-driven strategies that can help reduce traffic congestion and improve transportation systems.

Chapter 2

Methodology

Traffic congestion is raising a lot these days. Factors like expanding urban populations, uncoordinated traffic signal timing and a lack of real-time data. The effect of the traffic congestion is very huge these days. Data collected in this paper are from the Kaggle website for the implementations of machine learning algorithms using python to show outputs in the traffic prediction. Two datasets are collected in which one is the 2015's traffic data which comprises of date, time, and number of vehicles, junction and the rest one is the 2017's traffic data with the same details so as to compare easily without any misconception. The unwanted data has been deleted by pre-processing the data aggregated from 1 to 24 hours time interval to calculate traffic flow prediction with each 1 hour interval.

2.1 Data-description

2.1, Data Description

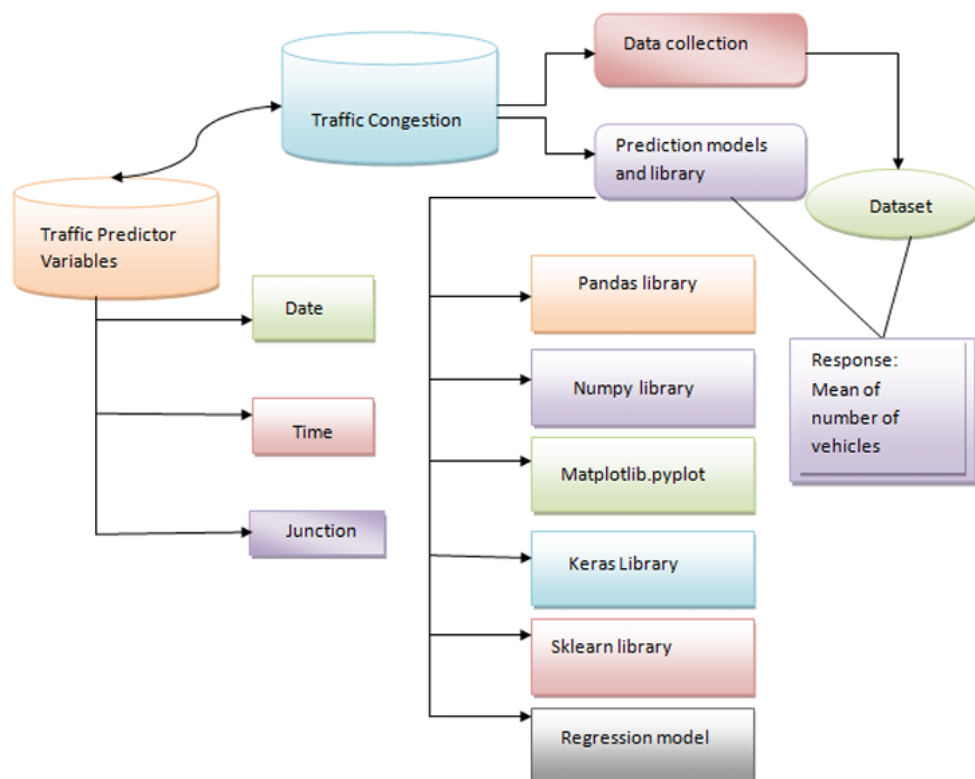
In traffic congestion forecasting there are data collection and prediction model.

	Junction	Vehicles	ID
count	48120.000000	48120.000000	4.812000e+04
mean	2.180549	22.791334	2.016330e+10
std	0.966955	20.750063	5.944854e+06
min	1.000000	1.000000	2.015110e+10
25%	1.000000	9.000000	2.016042e+10
50%	2.000000	15.000000	2.016093e+10
75%	3.000000	29.000000	2.017023e+10
max	4.000000	180.000000	2.017063e+10

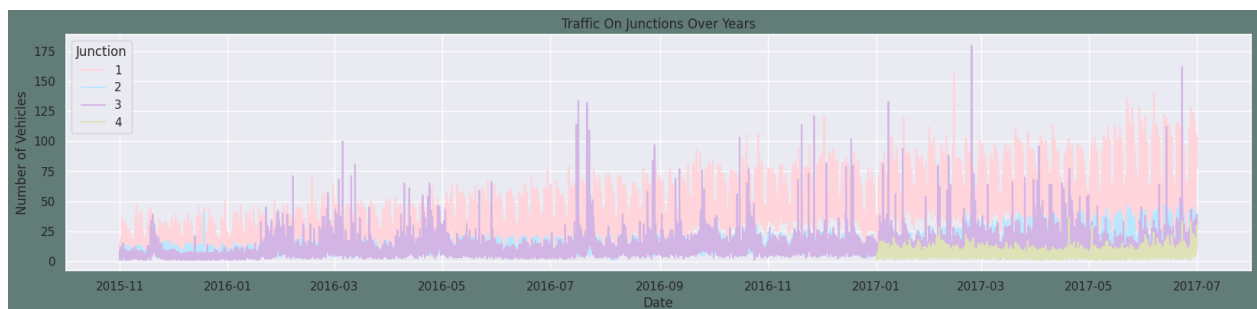
The traffic dataset used for this study consists of vehicle count data collected from various junctions at hourly intervals. The dataset includes the following columns:

	Vehicles			
Junction	1	2	3	4
count	14592.000000	14592.000000	14592.000000	4344.000000
mean	45.052906	14.253221	13.694010	7.251611
std	23.008345	7.401307	10.436005	3.521455
min	5.000000	1.000000	1.000000	1.000000
25%	27.000000	9.000000	7.000000	5.000000
50%	40.000000	13.000000	11.000000	7.000000
75%	59.000000	17.000000	18.000000	9.000000
max	156.000000	48.000000	180.000000	36.000000

Work-flow for analysis

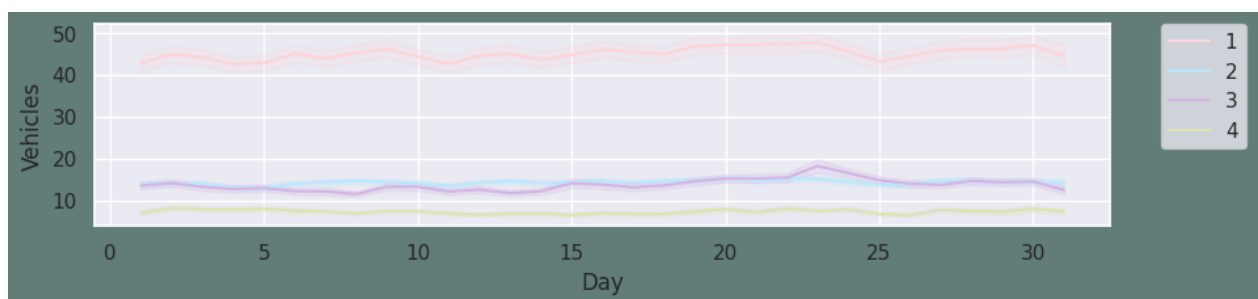
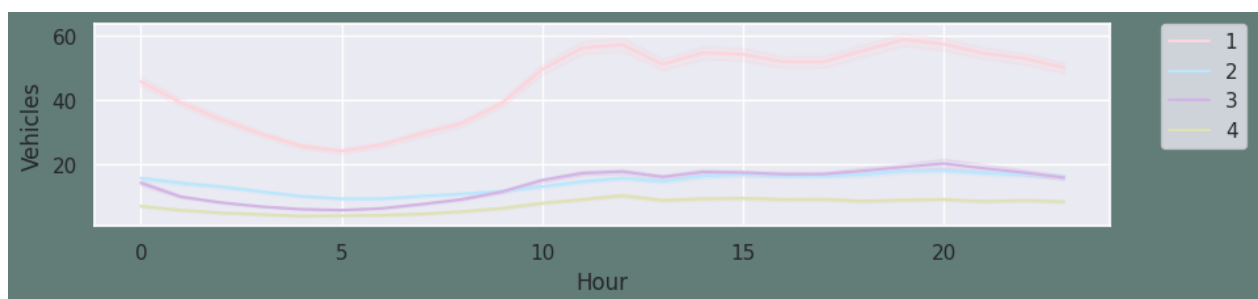
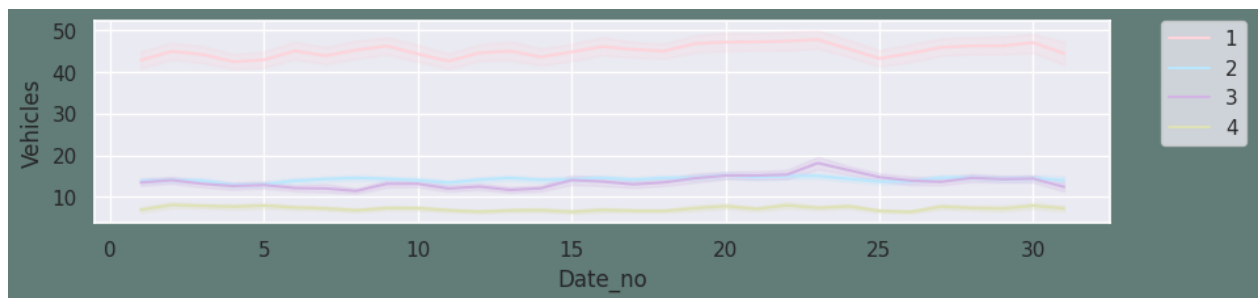
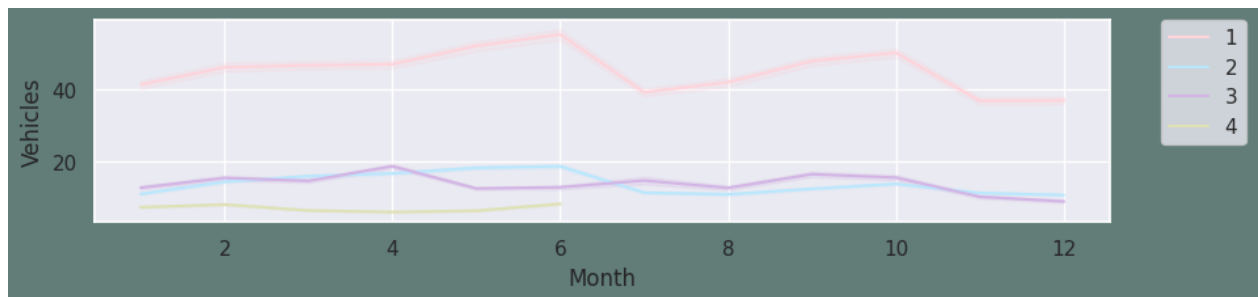
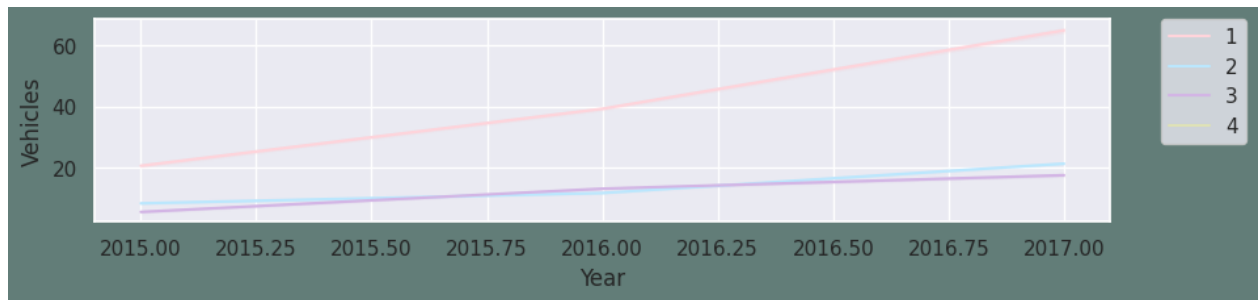


2.2 Exploratory-data-analysis



Feature Engineering

This is feature engineering after we drop the id new-features = ["Year","Month",
"Date-no", "Hour", "Day"]



2.2.1 Data Cleaning

Data cleaning is a key element in the preparation of our traffic data for analysis. We aim to see that after cleaning the data is accurate, that it is the same across the board, and

that it does not have any errors which may in turn present themselves in the results. The traffic data we are working with has many fields which must be handled with care to see that they are suitable for analysis. Also included below are the main steps we take in the cleaning of traffic data.

1, Missing Data:

There are no missing values in any of the columns (DateTime, Junction, Vehicles, ID), meaning the data is complete in terms of missing entries.

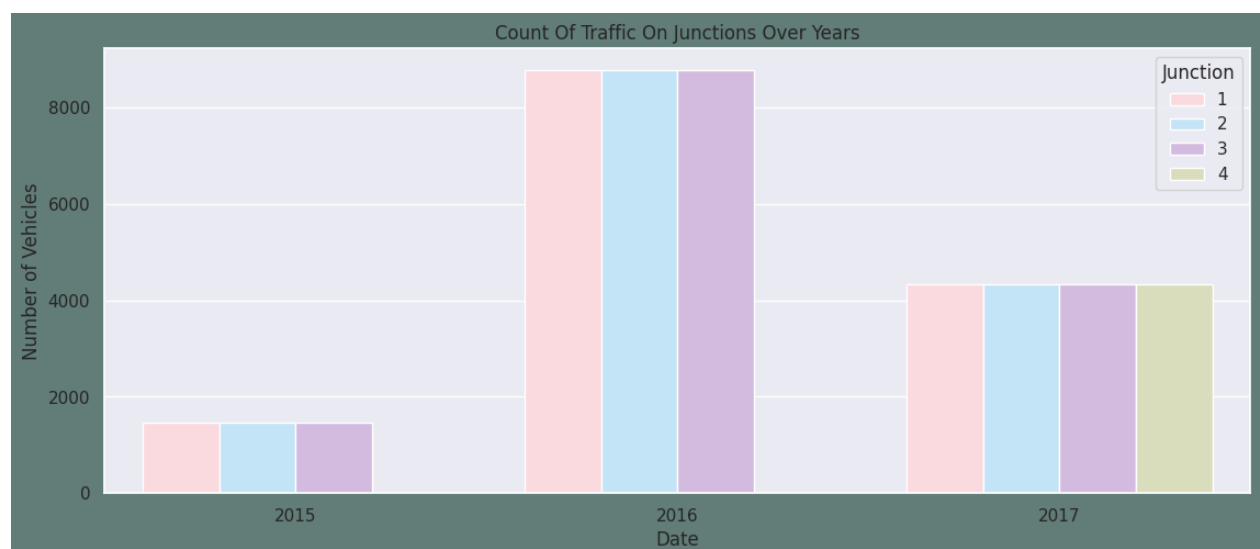
2, Duplicate Entries:

There are no duplicate rows in terms of DateTime, Junction, and ID, indicating that the dataset does not contain any repeated records.

3, Outliers:

The Vehicles column has a range from a minimum of 1 to a maximum of 180, with a mean of approximately 22.79 and a standard deviation of 20.75. While there are no extreme outliers apparent at a glance, there could be data points that are higher or lower than expected depending on the specific traffic flow patterns for each junction. A more detailed outlier detection method, like IQR (Interquartile Range) or z-scores, can further help identify unusual values.

4, Junction IDs:



There are 4 unique junctions in the data set, suggesting that traffic data from four different locations is being tracked.

2.2.2 Visualizing the data transformation

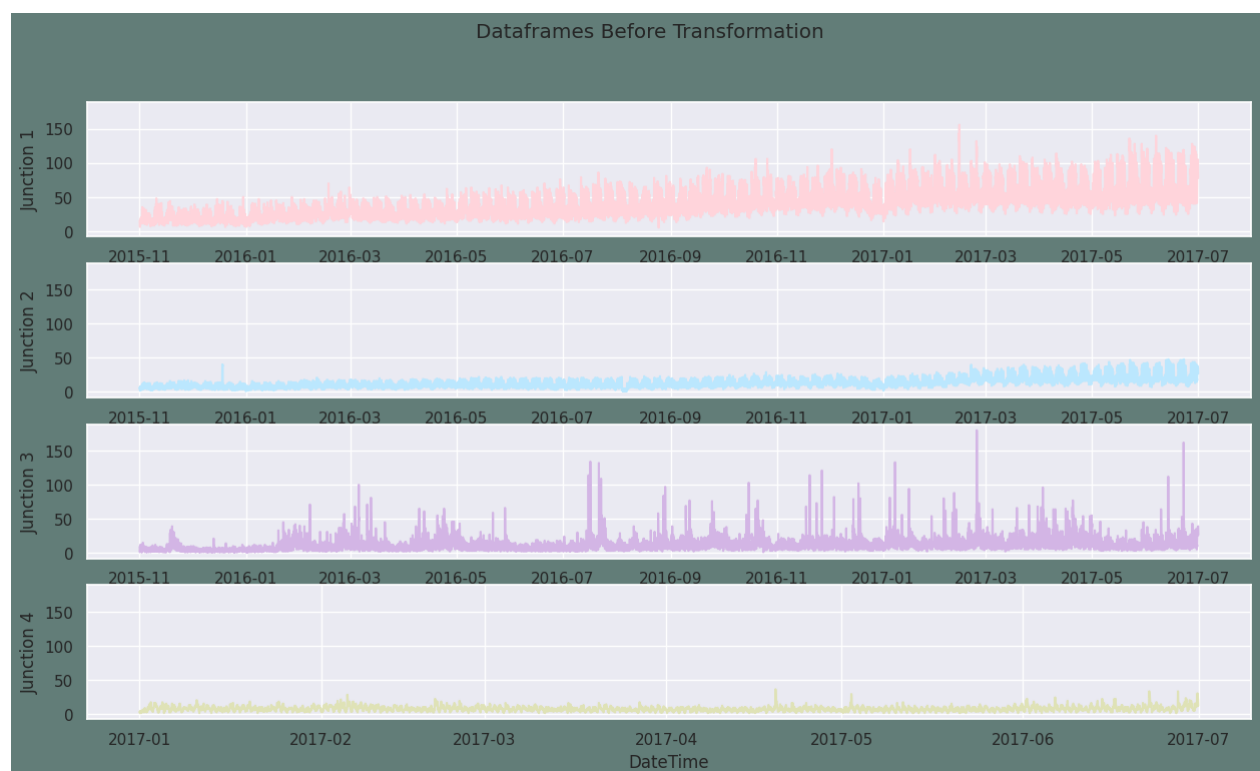
Why did we use data transformation in this project traffic prediction.?

Data preparation is a key step in the traffic data set that we present for analysis and predictive modeling. We do this to see that the data are in the right form and scale, which in turn makes it a better fit for machine learning models and also improves the accuracy of predictions. We look at the main transformation techniques that we applied to the data:

Normalization and Scaling:

Reason: Machine learning models, especially those that rely on distance calculations (like regression models or clustering), perform better when the data is normalized or scaled.

Implementation: Numerical features, such as the vehicle counts, may have varying scales (e.g., counts ranging from 1 to 180), so these values were scaled to ensure that no feature dominates the model due to its scale.



Encoding Categorical Variables:

Reason: Machine learning algorithms cannot process categorical variables directly. In this dataset, Junction is a categorical variable.

Implementation: The Junction column was converted into a numerical format using techniques like one-hot encoding or label encoding, which allow the algorithm to use it in predictive modeling.

Handling Outliers:

Reason: Outliers, such as unusually high vehicle counts, can distort model training by disproportionately affecting the results.

Implementation: Outliers in the Vehicles column were identified using basic descriptive statistics (e.g., boxplots or z-scores). Some extreme values were adjusted or removed to prevent them from skewing the analysis.

Feature Extraction from DateTime:

Reason: The DateTime column provides important temporal information that can help in understanding traffic patterns over time (e.g., daily, weekly, or seasonal trends).

Implementation: Additional features such as the hour of the day, day of the week, and month were extracted from the DateTime column to enhance the model's ability to capture time-dependent trends in traffic flow.

Log Transformation:

Reason: Some features, like the number of vehicles, may have skewed distributions that could affect the model's ability to learn meaningful patterns.

Implementation: A logarithmic transformation was applied to the Vehicles column to reduce skewness and make the distribution more normal, improving model accuracy.



Making the Data Stationary:

Reason: For time-series forecasting models, such as ARIMA, the data must be stationary (i.e., its statistical properties must remain constant over time).

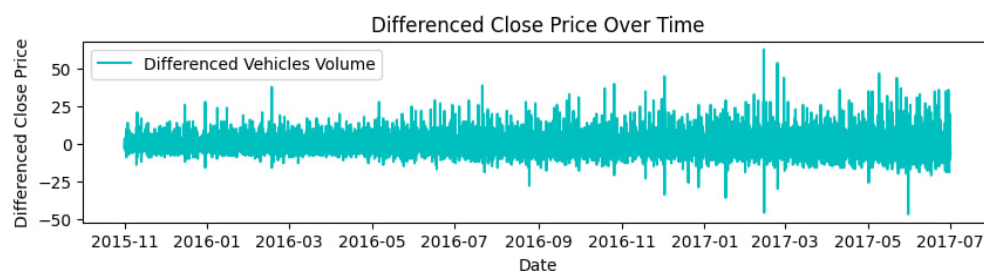
Implementation: Techniques like differencing and log transformation were applied to make the data stationary, enabling more accurate forecasting of future traffic patterns.

Checking the transformed series for stationarity for Junction 1:

ADF Statistic (Original): -7.1481

p-value (Original): 0.0000

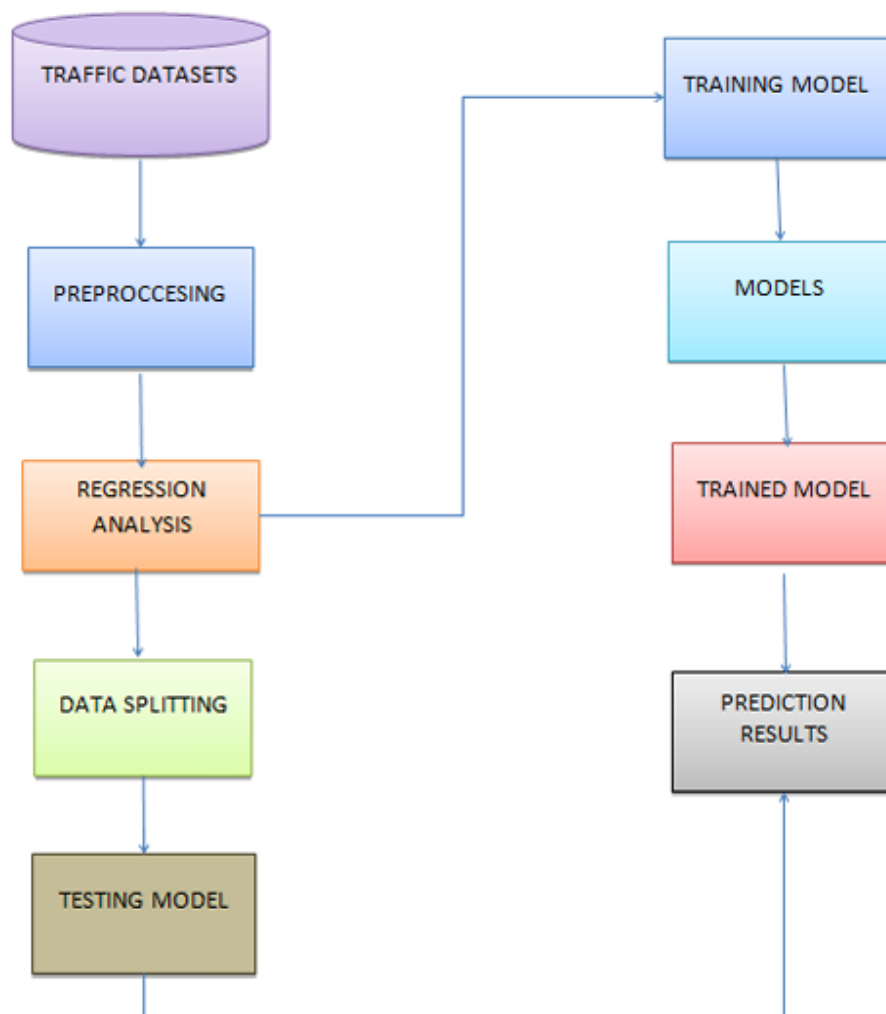
Interpretation: The original series is Stationary.



Chapter 3

Model Building

Regressor model analysis could even be a mathematical technique for resolving the connection in the middle of one dependent (criterion) variable and one or more independent (predictor) variables.



Train-Test Split

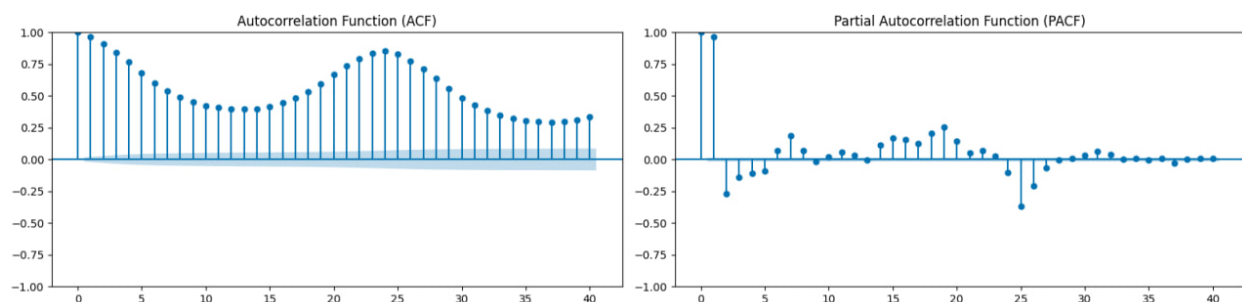
Train-Test Split The dataset was split into training and testing sets based on chronological order to preserve the temporal structure essential for time series forecasting. Data up to May 31, 2017 was used for training, while data from June 1 to June 30, 2017 was reserved for testing. This split ensures no information leakage from future observations into model training and allows for an unbiased evaluation of forecasting performance.

3.1 Model Arima and Sarima

ARIMA: A time series forecasting method that models data using autoregression, differencing, and moving averages to capture trends and patterns.

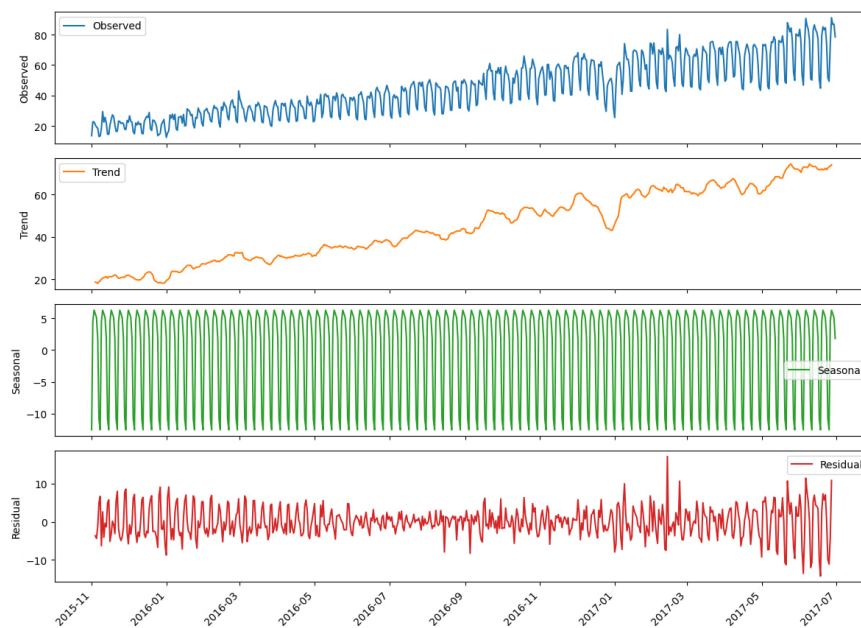
1. Parameter Identification:

We analyzed the time series using ACF (Autocorrelation Function) and PACF (Partial Autocorrelation Function) plots to identify suitable ARIMA parameters. Alternatively, the auto-arima function was used for automated parameter selection. This step helped determine the values for (p, d, q) and seasonal (P, D, Q, s) components.



Decomposition is the process of breaking down a time series into its fundamental components typically trend, seasonality, and residual (or noise). This helps in understanding the underlying patterns by isolating systematic changes (trend), repeating seasonal effects, and random fluctuations. In simpler terms, decomposition separates a complex time series into easier-to-analyze parts to identify and model each component individually.

Decomposition of Weekly Traffic



Non-stationary 0.9194355462710133

Training set and test set

Split into train (up to May 31, 2017) and test (June 2017)

```
train_data = junction_1_data['2015-11-01':'2017-02-28']['Vehicles']
test_data = junction_1_data['2017-03-01':'2017-06-30']['Vehicles']
```

2. Model Fitting:

A Seasonal ARIMA (SARIMA) model was fit to the time series data using the identified parameters. This model captures both trend and seasonality in the dataset.

Grid search for finding the best parameters.

```
Order=(0, 0, 0), Seasonal=(0, 0, 0, 24), AIC=121348.31650912951
Order=(0, 0, 0), Seasonal=(0, 0, 1, 24), AIC=108689.17790810147
Order=(0, 0, 0), Seasonal=(1, 0, 0, 24), AIC=89285.343078983
Order=(0, 0, 0), Seasonal=(1, 0, 1, 24), AIC=86586.02419958072
c:\Users\ASUS\AppData\Local\Programs\Python\Python312\Lib\site-packages\statsmodels\tsa\statespace\sarimax.py:978: UserWarning: Non-invertible starting MA parameters found. Using zeros as starting parameters:
warn('Non-invertible starting MA parameters found.')
Order=(0, 0, 1), Seasonal=(0, 0, 0, 24), AIC=107118.95489792924
c:\Users\ASUS\AppData\Local\Programs\Python\Python312\Lib\site-packages\statsmodels\tsa\statespace\sarimax.py:978: UserWarning: Non-invertible starting MA parameters found. Using zeros as starting parameters:
warn('Non-invertible starting MA parameters found.')
Order=(0, 0, 1), Seasonal=(0, 0, 1, 24), AIC=97351.02707234374
c:\Users\ASUS\AppData\Local\Programs\Python\Python312\Lib\site-packages\statsmodels\tsa\statespace\sarimax.py:978: UserWarning: Non-invertible starting MA parameters found. Using zeros as starting parameters:
warn('Non-invertible starting MA parameters found.')
Order=(0, 0, 1), Seasonal=(1, 0, 0, 24), AIC=81138.2175404168
c:\Users\ASUS\AppData\Local\Programs\Python\Python312\Lib\site-packages\statsmodels\tsa\statespace\sarimax.py:978: UserWarning: Non-invertible starting MA parameters found. Using zeros as starting parameters:
warn('Non-invertible starting MA parameters found.')
Order=(0, 0, 1), Seasonal=(1, 0, 1, 24), AIC=77282.37180844528
Order=(1, 0, 0), Seasonal=(0, 0, 0, 24), AIC=73689.5416319629
Order=(1, 0, 0), Seasonal=(0, 0, 1, 24), AIC=71494.1830185457
Order=(1, 0, 0), Seasonal=(1, 0, 0, 24), AIC=70805.42947754143
Order=(1, 0, 0), Seasonal=(1, 0, 1, 24), AIC=65532.60455713473
Order=(1, 0, 1), Seasonal=(0, 0, 0, 24), AIC=73351.56090966964
Order=(1, 0, 1), Seasonal=(0, 0, 1, 24), AIC=71485.0690492226
Order=(1, 0, 1), Seasonal=(1, 0, 0, 24), AIC=69787.06936847651
Order=(1, 0, 1), Seasonal=(1, 0, 1, 24), AIC=64921.63580183587

Best Parameters: order=(1, 0, 1), seasonal_order=(1, 0, 1, 24)
Best AIC: 64921.63580183587
```

Order=(0, 0, 0), Seasonal=(0, 0, 0, 24), AIC=121348.31650912951

Order=(0, 0, 0), Seasonal=(0, 0, 1, 24), AIC=108689.17790810147

Order=(0, 0, 0), Seasonal=(1, 0, 0, 24), AIC=89205.343078983

Order=(0, 0, 0), Seasonal=(1, 0, 1, 24), AIC=86506.02419958072

c:312-packages.py:978: UserWarning: Non-invertible starting MA parameters found. Using zeros as starting parameters.

warn('Non-invertible starting MA parameters found.')

Order=(0, 0, 1), Seasonal=(0, 0, 0, 24), AIC=107118.95489792924

c:312-packages.py:978: UserWarning: Non-invertible starting MA parameters found. Using zeros as starting parameters.

warn('Non-invertible starting MA parameters found.')

Order=(0, 0, 1), Seasonal=(0, 0, 1, 24), AIC=97351.02707234374

c:312-packages.py:978: UserWarning: Non-invertible starting MA parameters found. Using zeros as starting parameters.

warn('Non-invertible starting MA parameters found.')

Order=(0, 0, 1), Seasonal=(1, 0, 0, 24), AIC=81138.2175404168 c:312-packages.py:978: UserWarning: Non-invertible starting MA parameters found. Using zeros as starting parameters.

warn('Non-invertible starting MA parameters found.')

Order=(0, 0, 1), Seasonal=(1, 0, 1, 24), AIC=77282.37180844528

Order=(1, 0, 0), Seasonal=(0, 0, 0, 24), AIC=73689.5416319629

Order=(1, 0, 0), Seasonal=(0, 0, 1, 24), AIC=71494.1830105457

Order=(1, 0, 0), Seasonal=(1, 0, 0, 24), AIC=70005.42947754143

Order=(1, 0, 1), Seasonal=(0, 0, 0, 24), AIC=73351.56090966964

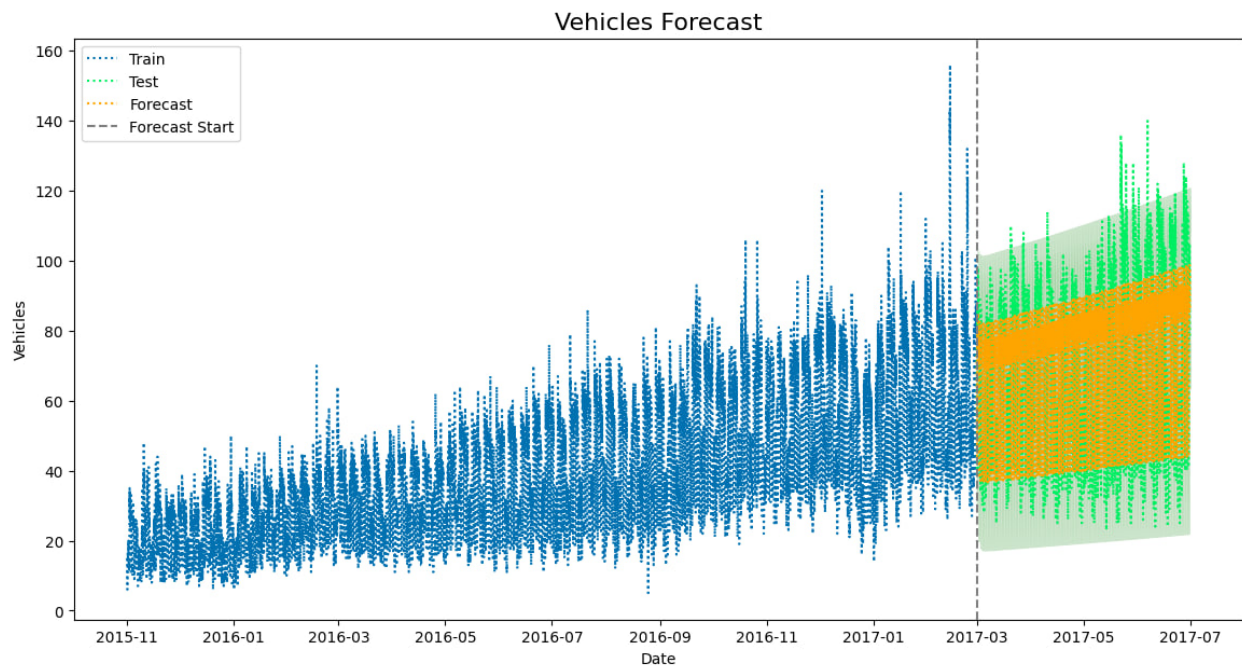
Order=(1, 0, 1), Seasonal=(0, 0, 1, 24), AIC=71485.0690492226

Order=(1, 0, 1), Seasonal=(1, 0, 0, 24), AIC=69787.06936847651

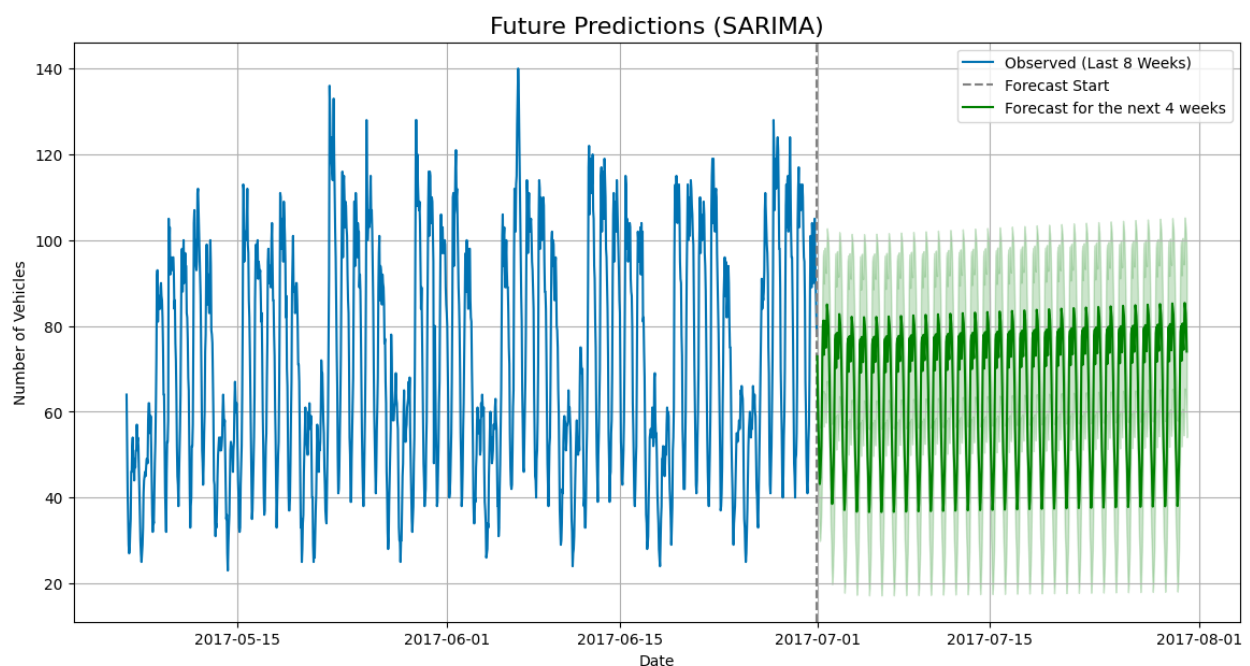
Order=(1, 0, 1), Seasonal=(1, 0, 1, 24), AIC=64921.63580183587

Best Parameters: order=(1, 0, 1), seasonal-order=(1, 0, 1, 24)

```
final_model = SARIMAX(  
    train_data,  
    order=(best_params[0], d, best_params[1]),  
    seasonal_order=(best_params[2], D, best_params[3], s),  
    enforce_stationarity=False,  
    enforce_invertibility=False  
)  
results = final_model.fit()
```

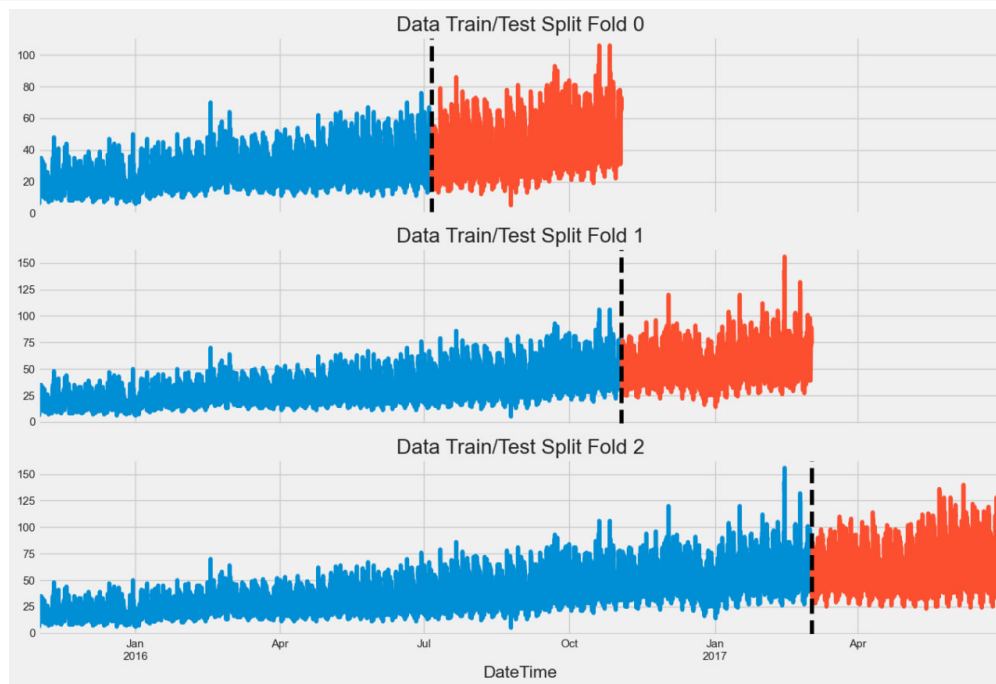



Future prediction of SARIMA Model



3.2 XGBoost and advantage XGBoost

XGBoost: A powerful, efficient gradient boosting algorithm that builds an ensemble of decision trees to improve prediction accuracy, widely used for regression and classification tasks.



Used TimeSeriesSplit for 3-fold time-based cross-validation. Created time features (hour, day, month, etc.) to train an XGBoost regression model predicting vehicle counts. Evaluated performance using RMSE on each test fold, ensuring proper temporal validation and feature use.

```
tss = TimeSeriesSplit(n_splits=3, test_size=24*120*1, gap=24)
j_1 = j_1.sort_index()

fold = 0
preds = []
scores = []
for train_idx, val_idx in tss.split(j_1):
    train = j_1.iloc[train_idx]
    test = j_1.iloc[val_idx]

    train = create_features(train)
    test = create_features(test)

    FEATURES = ['hour', 'dayofweek', 'month', 'year', 'dayofyear', 'dayofmonth', 'weekofyear']
    TARGET = 'Vehicles'

    X_train = train[FEATURES]
    y_train = train[TARGET]

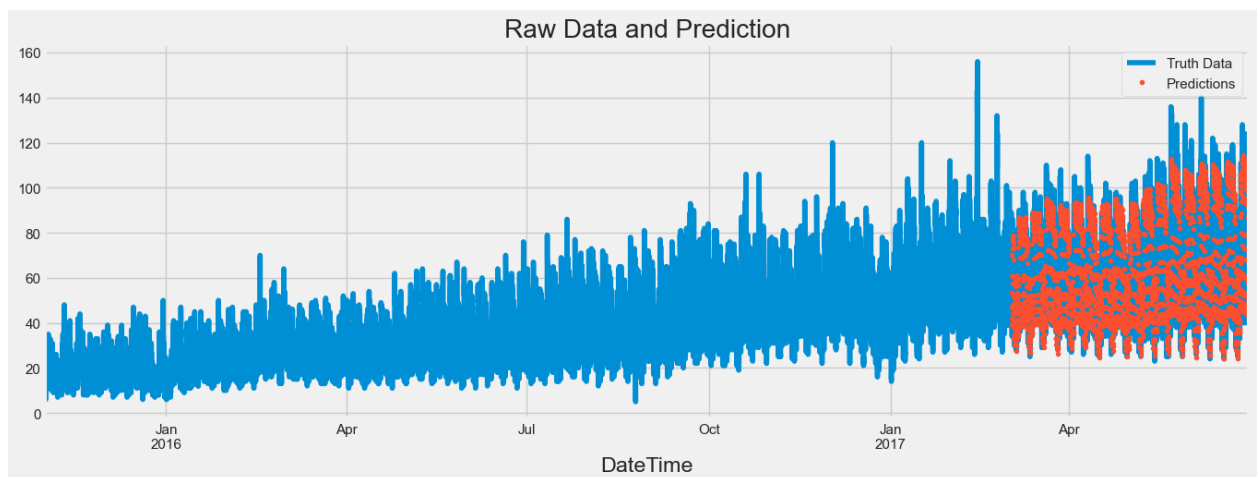
    X_test = test[FEATURES]
    y_test = test[TARGET]

    reg = xgb.XGBRegressor(base_score=0.5, booster='gbtree',
                           n_estimators=1000,
                           early_stopping_rounds=50,
                           objective='reg:linear',
                           max_depth=3,
                           learning_rate=0.01)

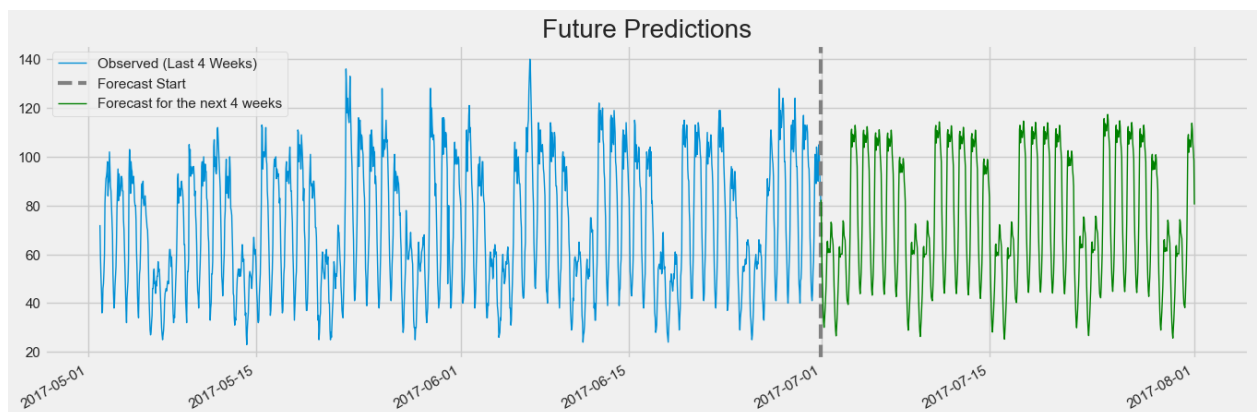
    reg.fit(X_train, y_train,
            eval_set=[(X_train, y_train), (X_test, y_test)],
            verbose=100)

    y_pred = reg.predict(X_test)
    preds.append(y_pred)
    score = np.sqrt(mean_squared_error(y_test, y_pred))
    scores.append(score)
```

The plot shows the historical truth data (blue line) and the model's predictions (red dots) over time. The model captures the overall trend and seasonality well, though predictions appear smoother and less volatile than the actual data. The forecast period begins around early 2017, demonstrating the model's ability to predict future values with reasonable accuracy.



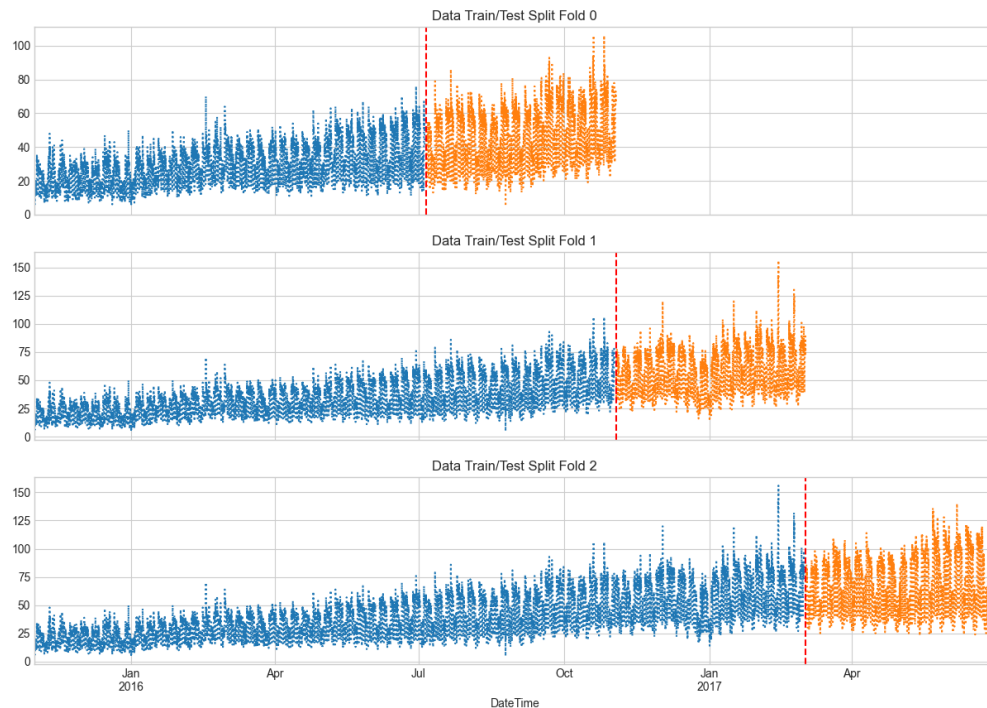
Future prediction of XGBoost model



3.3 Random-forest Model

Random Forest: An ensemble learning method using multiple decision trees to improve prediction accuracy by averaging (regression) or voting (classification), applicable to time series with appropriate feature.

This graph was show about train and test split of the data:



Train/Test Split Visualization

The figure displays the time-based splits of the dataset into training and test sets across 3 folds using TimeSeriesSplit. Each subplot shows the training data (dotted line) and corresponding test data (dotted line) for a fold, with a vertical red dashed line marking the start of the test period. This visualization confirms that the splits respect temporal order, avoiding data leakage and ensuring realistic validation.

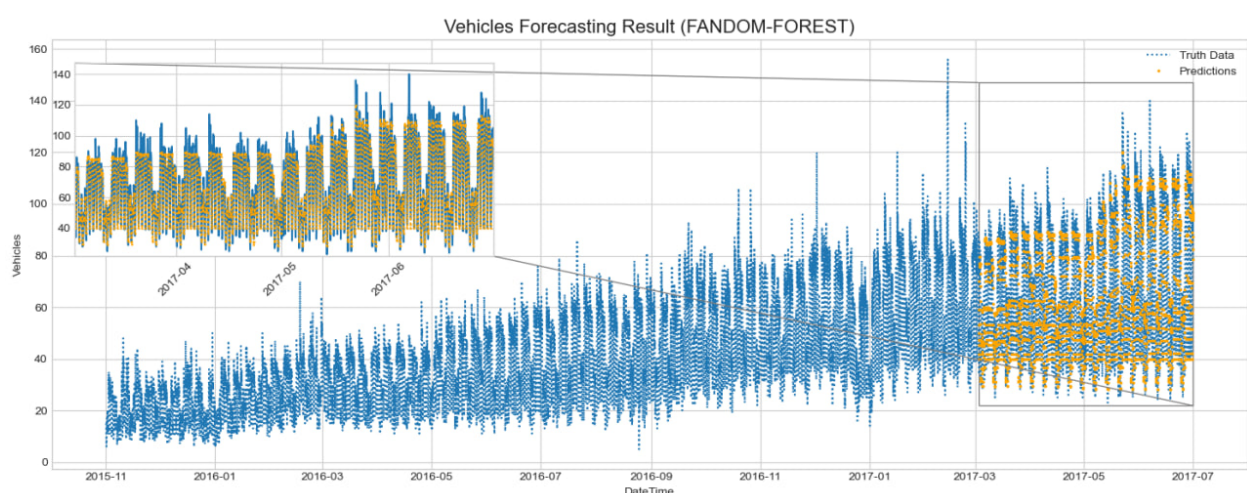
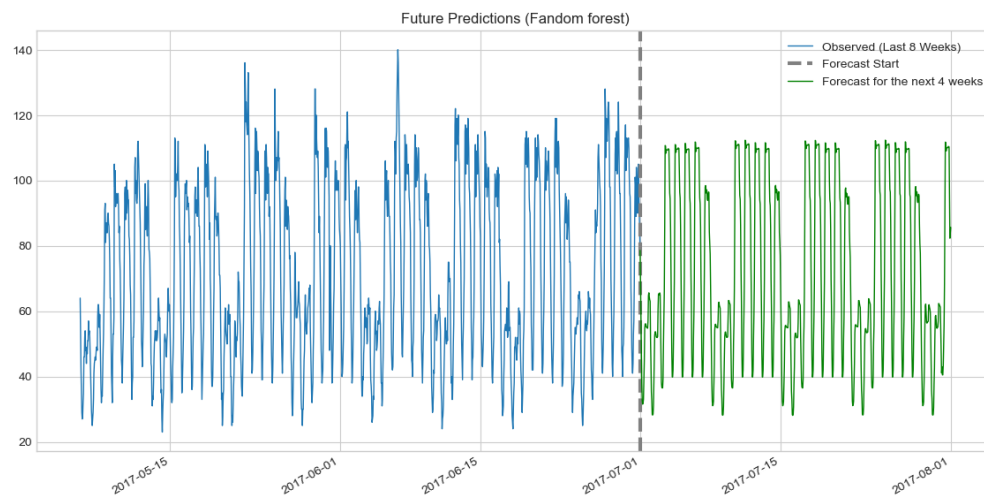


Figure X shows how well the Fandom forest model predicts the number of vehicles each hour. The orange dots are the model's predictions, and the blue line is the real data. The zoomed-in part highlights the test period (June 2017), where the predictions follow

the actual trend closely.

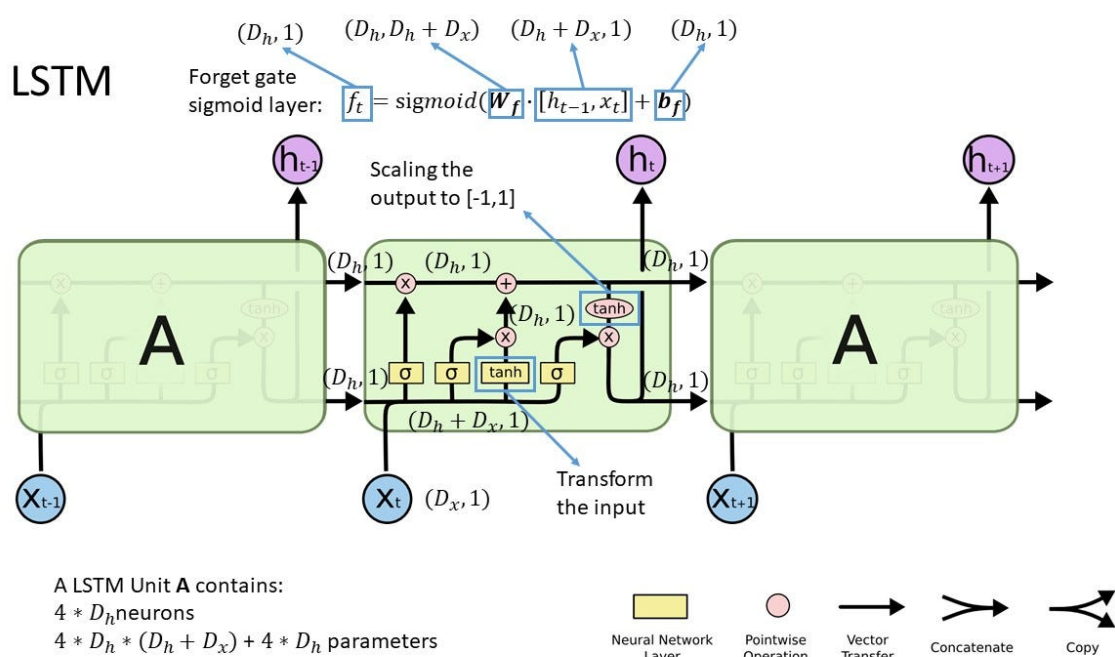
Future prediction of RANDOM FOREST



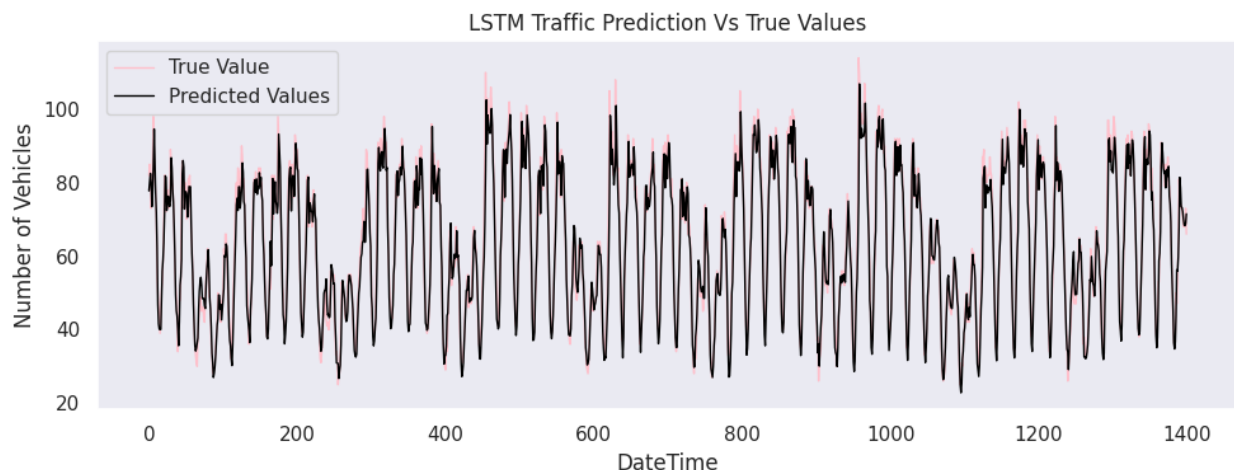
3.4 LSTM Model deep learning

Long short-term memory (LSTM) is a type of recurrent neural network (RNN) aimed at mitigating the vanishing gradient problem commonly encountered by traditional RNNs. Its relative insensitivity to gap length is its advantage over other RNNs, hidden Markov models, and other sequence learning methods. It aims to provide a short-term memory for RNN that can last thousands of time steps.

Concept of LSTM



Here is graph that predicted by using model LSTM



Training Details The LSTM model was trained to predict hourly values from the time series data. Input sequences of past hourly readings were used to forecast the next hour.

- Data was normalized, and sequences of fixed length (e.g., 24 hours) were created.
- The model had one/two LSTM layers with [e.g., 50] units, followed by a dense output layer.
- Training used MSE loss and the Adam optimizer with a learning rate of 0.001.
- The model trained for [e.g., 50] epochs with batch size [e.g., 32].
- Early stopping monitored validation loss to avoid overfitting.
- Performance was measured with RMSE and MAE on the test set.

3.5 GRU Model

For this project, I have settled to use Gated Recurrent Unit (GRU). In this section, I am creating a function for the neural net to call on and fit the data frames for all four junctions.

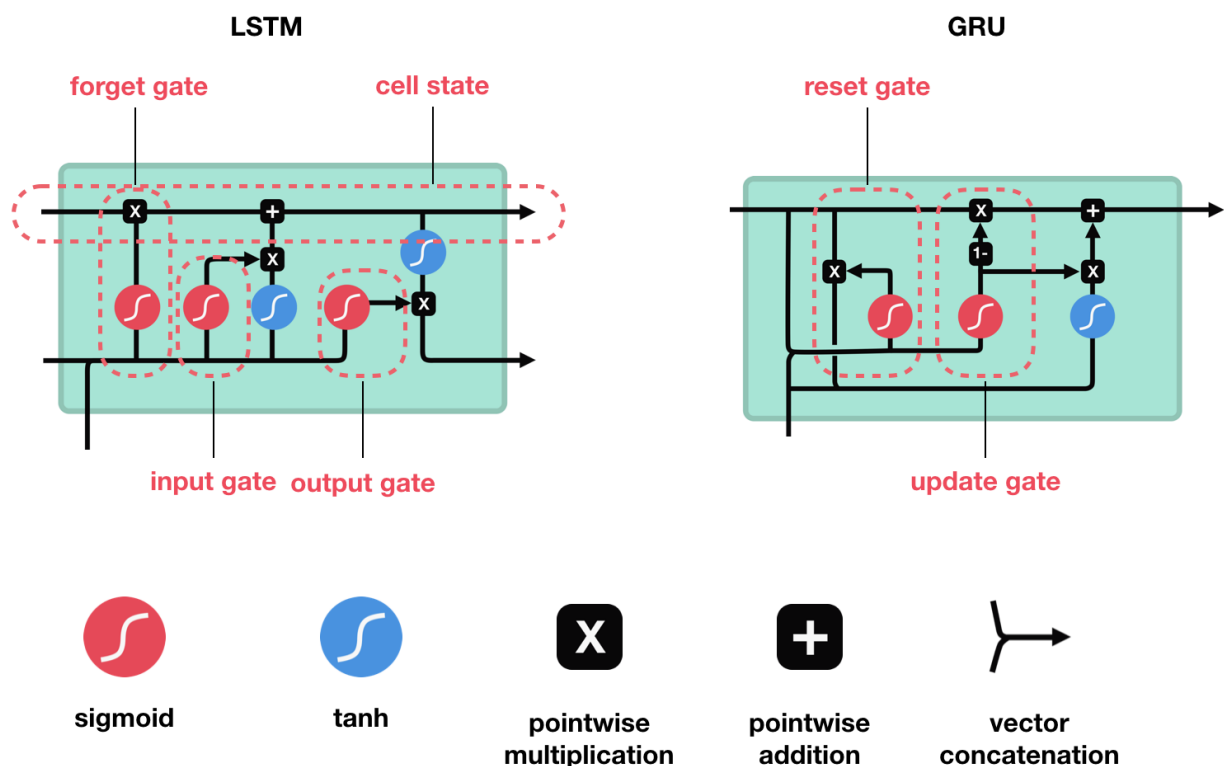
GRU (Gated Recurrent Unit) is a type of recurrent neural network that uses gating mechanisms to control information flow, allowing it to capture dependencies in sequential data efficiently with fewer parameters than LSTM.

Training Details (GRU)

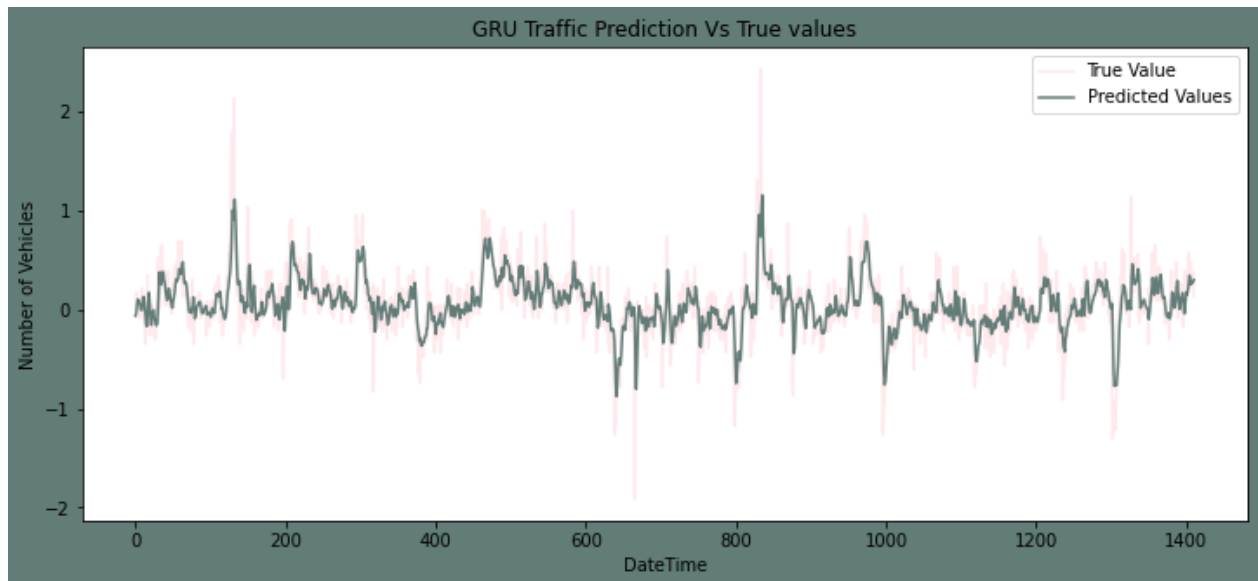
The GRU model was designed to predict hourly values using past hourly sequences.

- Data was normalized and segmented into fixed-length input sequences (e.g., 24 hours).
- The model consisted of one/two GRU layers with [e.g., 50] units, followed by a dense output layer.
- Training used Mean Squared Error (MSE) loss and the Adam optimizer at 0.001 learning rate.
- Trained for [e.g., 50] epochs with batch size [e.g., 32].
- Early stopping was applied based on validation loss to prevent overfitting.
- Evaluation metrics included RMSE and MAE on test data.

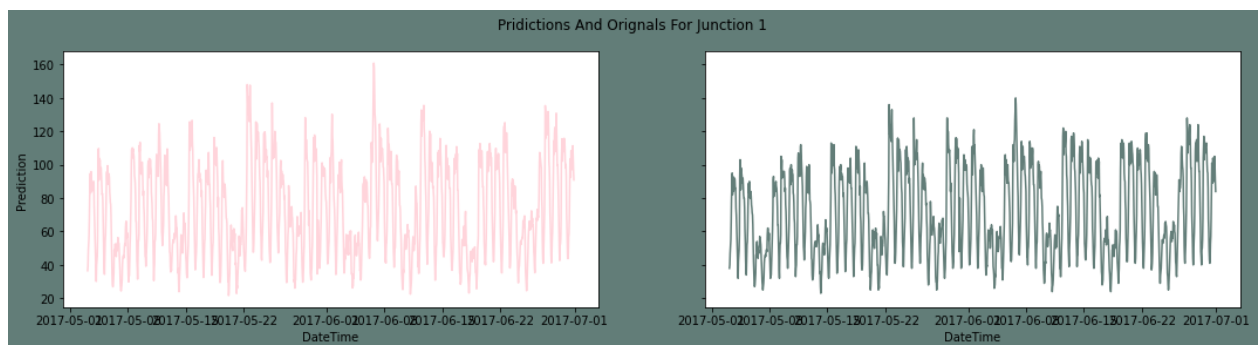
work-flow of GRU Model



Fitting the first junction and plotting the predictions and testset.



In this section, I will be inversing transforms that I applied to the datasets to remove the seasonality and trends. Performing this step will make the predictions get back on the accurate scale.



Chapter 4

Evaluation and Result

Model	MAE	MSE	RMSE	R^2 Score
SARIMA	11.8184	238.7529	15.4516	0.5715
XGBOOST	3.6918	26.3304	5.1313	0.9532
RANDOM_FOREST	3.3824	22.9632	4.792	0.6591
LSTM	4.6437	42.0731	6.4864	0.9246
GRU	3.89	29.2206	5.4056	0.9477

The error metrics MAE, MSE, RMSE, and the R^2 score provide a clear comparison of model performance on the hourly prediction task.

-MAE measures average prediction error magnitude. XGBoost (3.69) and Random Forest (3.38) show the lowest errors, indicating more accurate predictions on average.

-MSE and RMSE penalize larger errors more; Random Forest has the lowest MSE (22.96) and RMSE (4.79), highlighting its ability to minimize big mistakes.

- R^2 score evaluates how well the model explains variance. GRU (0.9477) and XGBoost (0.9532) outperform others, capturing over 94

-SARIMA performs worst across all metrics, indicating it's less suitable for this dataset.

Overall, tree-based models (XGBoost, Random Forest) and GRU show superior accuracy and explanatory power, with GRU balancing strong R^2 and low error metrics, validating its suitability for hourly forecasting on this dataset.

4.1 Reference

Appendix - Python code

- A. [source of dataset and GRU model building](#)
- B. [source of model arima](#)
- C. [source of XGBoost](#)
- D. [source of random forest model building](#)
- E. [source of LSTM model building](#)